Mixture of Memory: Tool-Aware KV Cache Retention for Agentic LLM Serving

Dakshinamoorthy A*, Alexander Ryssdal-Banoun†, Andrew Lee‡, Vatsalam Krishna Jha§
COMS E6998, High Performance Machine Learning, Columbia University, Spring 2026
{da3232, ar4678, al4744, vkj2107}@columbia.edu

***Abstract*—Multi-turn agentic LLM workloads, where models pause to call external tools and then resume generation, suffer from expensive KV cache recomputation on every tool return. We implement three complementary inference-time optimizations on top of vLLM v0.6.4.post1: (1) a tool-aware KV retention policy that uses an EMA-based TTL predictor to pin KV blocks during tool gaps, avoiding unnecessary reprefill; (2) INT8/INT4 quantization of cached KV states with selective full-precision retention for recent turns; and (3) `torch.compile` integration with phase-tagged profiling infrastructure. In sequential isolation, retention achieves a 10.2× post-tool TTFT speedup on an L4 GPU and 4.2× on an H100, with 100% pin survival under 60-agent concurrent load. INT4 KV quantization reduces active working-set memory by 4× versus FP16 with no measurable per-token latency cost. Retention preserves task accuracy at FP16 with zero measurable degradation on GSM8K and BBH ($n = 50$ per subtask). A two-point cache-pressure characterization shows that retention's value depends on the cache regime: under abundant cache it adds overhead without compensating benefit, while under constrained cache it reduces p95 post-tool TTFT by 28% relative to prefix caching alone. All code is open-source at https://github.com/al4744/MoM.**

I. Introduction

Large language models are increasingly deployed as agentic systems that interleave generation with external tool calls (database lookups, API requests, code execution) across multi-turn conversations [2], [3]. Each tool call introduces a pause during which the model's KV cache, holding the accumulated context, sits idle on the GPU. Under memory pressure from concurrent requests, the serving engine may evict these cached states. When the tool returns and generation resumes, the engine must recompute the full prefill from scratch, incurring latency proportional to the context length.

This recomputation is the dominant latency bottleneck in multi-turn agentic serving. In our baseline measurements on a single L4 GPU with Llama-3-8B, post-tool time-to-first-token (TTFT) reaches 843 ms per tool return: time spent entirely on redundant KV recomputation for context the model has already processed.

Existing KV cache management in vLLM [1] treats all sequences identically: eviction is driven by LRU ordering and block reference counts, with no awareness of whether a sequence is paused for a short tool call or has been abandoned. Prefix caching [1] can partially mitigate recomputation by matching cached block hashes, but it provides no mechanism to protect specific blocks from eviction during tool gaps.

We present three complementary optimizations implemented on top of a vendored copy of vLLM v0.6.4.post1:

1) **Tool-aware KV retention** (Workstream A): A pin-or-evict policy with an EMA-based per-tool TTL predictor that holds KV blocks during predicted tool-call durations, enabling zero-cost cache reuse on tool return.
2) **KV cache quantization** (Workstream B): INT8 and INT4 symmetric quantization of cached KV states (not model weights) with selective full-precision retention for recent conversation turns.
3) **`torch.compile` integration** (Workstream C): Compilation of the vLLM inference path via `VLLM_TORCH_COMPILE_LEVEL` with phase-tagged profiling instrumentation using PyTorch Profiler and NVIDIA Nsight Systems.

A central methodological contribution of this work is the identification of conditions under which retention demonstrates measurable value, validated across four distinct operating regimes. The optimizations target inference-time serving latency and memory efficiency. No model weights are modified, and no training is performed. The evaluation uses synthetic multi-agent traces with controlled tool-call patterns to isolate the pause-resume bottleneck from confounding prompt-level variation.

II. Literature Review

PagedAttention and vLLM. Kwon et al. [1] introduced PagedAttention, which manages KV cache as fixed-size blocks in a virtual memory system, enabling non-contiguous storage and efficient sharing. vLLM's block manager handles allocation, eviction, and prefix caching. Our work extends this block-level abstraction with a pin mechanism that defers block deallocation for sequences in tool gaps.

KV cache eviction policies. H_2O [4] identifies "heavy hitter" attention heads whose KV entries contribute disproportionately to output quality, evicting low-attention entries to reduce memory. Scissorhands [5] prunes KV entries based on attention persistence across decoding steps. Both operate at the token level within a single sequence. Our retention policy operates at the block level across sequences, deciding which paused agent's cache to protect from cross-agent eviction pressure.

KV cache compression. KIVI [6] applies per-channel INT2 quantization to KV cache with residual-length grouping. KVQuant [7] uses per-channel key quantization and per-token value quantization with non-uniform datatypes. Our

INT8/INT4 implementation uses symmetric per-block quantization with a selective policy: recent turns retain full precision while older context is quantized, balancing memory savings against generation quality.

Memory-augmented LLM serving. MemGPT [8] introduces an OS-inspired virtual memory hierarchy for LLM context, paging context between main memory and a limited context window. LMCache provides tiered KV caching across GPU, CPU, disk, and remote storage. Our approach is complementary: rather than building an external caching tier, we modify vLLM's internal block lifecycle to retain KV blocks in-place on GPU during predicted tool gaps.

Torch.compile for inference. PyTorch 2.x's `torch.compile` [9] applies graph-level optimizations including operator fusion and memory planning. In vLLM v0.6.4, compilation is exposed at the model execution level through an environment variable. Our integration instruments this path with phase-tagged profiler ranges to separately attribute prefill and decode performance.

III. Methodology

A. Base Framework and Models

All optimizations are built on a vendored fork of vLLM v0.6.4.post1 (November 2024). This version was selected because (a) it lacks built-in KV offloading connectors, allowing us to build the retention pipeline from scratch; (b) `torch.compile` support is experimental, enabling measurable gains; and (c) the V0 engine architecture is simpler to modify than the V1 engine introduced in v0.8.0+.

The primary model is **Meta-Llama-3-8B** in FP16, with a maximum context length of 8,192 tokens. The `gpu_memory_utilization` parameter is set to 0.85 for standard runs and 0.30 for cache-constrained experiments, the two operating points used to establish the cache-pressure characterization in §IV-G.

B. Tool-Aware KV Retention (Workstream A)

The retention system consists of two components integrated into vLLM's scheduler loop. Figure 1 shows the end-to-end KV block lifecycle across a single tool-call gap.

TTL Predictor. An exponential moving average (EMA) estimator that predicts how long a tool call will take, producing a time-to-live (TTL) value for pinned KV blocks. The predictor maintains per-tool EMAs (updated with smoothing factor $\alpha = 0.3$) and falls back to a global EMA or a configurable default ($\tau_0 = 1.0\,\text{s}$) for unseen tools. The predicted TTL is scaled by a safety factor ($\sigma = 1.5$) to cover tool-call jitter:

$$\text{TTL}(t) = \sigma \cdot \text{EMA}(t) \tag{1}$$

Pin Manager. When vLLM's scheduler finishes a sequence whose next turn is a tool call, instead of freeing the sequence's KV blocks, the scheduler hands the `SequenceGroup` reference to the PinManager. The held reference keeps block reference counts above zero, preventing eviction. The PinManager enforces a configurable budget (`max_pinned_fraction` = 0.30 of total GPU blocks) and implements three release paths:

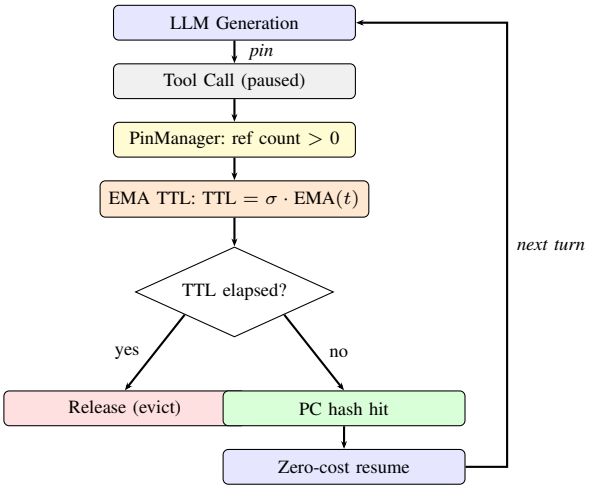

Fig. 1. Tool-aware KV retention lifecycle. The PinManager holds KV block reference counts above zero during a tool gap. The EMA TTL predictor sets a per-tool deadline. If TTL elapses before the agent returns (yes), blocks are released. If the agent returns in time (no), a prefix-cache hash hit delivers zero-cost resume.

- **Reuse**: The next turn of the same agent arrives before TTL expiry. The PinManager releases the held reference; vLLM's prefix-cache hash lookup finds the blocks in the evictor and revives them at zero reprefill cost.
- **Expiry**: TTL elapses without a returning request. Blocks are freed normally.
- **Eviction**: Under memory pressure, the soonest-expiring pinned entry is evicted as a pressure valve.

Structured events (`pin`, `reuse`, `expire`, `evict`, `pin_rejected_budget`) are logged to JSONL for post-hoc analysis. Retention requires prefix caching to be enabled: the `LLMEngine` raises a `ValueError` if retention is activated without prefix caching, since the pin-reuse mechanism depends on prefix-cache hash matching to revive held blocks.

C. KV Cache Quantization (Workstream B)

We implement symmetric per-block quantization for both INT8 and INT4 precision:

INT8: Each KV block tensor is scaled to the $[-127, 127]$ range using the maximum absolute value as the scale factor. Dequantization multiplies the stored `int8` payload by the scale.

INT4: Values are scaled to $[-7, 7]$, shifted to $[1, 15]$, and packed as nibble pairs into `uint8` tensors, halving storage relative to INT8. A per-block scale and zero-point are stored alongside the payload.

A selective retention policy preserves full FP16 precision for the most recent k turns (`recent_turns_fp` = 2 by default) while quantizing older context. This balances memory savings against quality for the tokens most likely to influence the next generation step. The quantization module is integrated into vLLM's `CacheEngine` via hooks in the vendored `cache_engine.py` and `llm_engine.py`.

D. Torch.compile and Profiling (Workstream C)

Compilation is activated by setting `VLLM_TORCH_COMPILE_LEVEL=1` in the environment before engine initialization. The `inductor` backend is used with default optimization mode.

Limitation: vLLM v0.6.4 exposes `torch.compile` at the model execution level: prefill and decode are not independently compiled as separate callables. Phase attribution is achieved through profiler ranges (`torch.profiler.record_function`) tagged as `mom.vllm.prefill`, `mom.vllm.decode`, and `mom.vllm.mixed`. Profiling infrastructure supports PyTorch Profiler (CPU + CUDA activity tracing with TensorBoard export) and NVIDIA Nsight Systems (via generated `nsys profile` command scripts).

E. Evaluation Framework (Workstream D)

Execution modes. The evaluation runner (`evaluation/run_eval.py`) supports three modes: dry-run (sentinel values for pipeline testing), mock (CPU-only `MockEngine` for smoke testing), and real (vendored vLLM with GPU).

Synthetic traces. Workloads are parameterized by turn count (5/10/25/50), tool-call frequency, tool latency distribution, and arrival pattern. Five workload classes target distinct contention regimes:

- **Lockstep**: N identical agents starting simultaneously (negative control, no cross-agent contention).
- **Filler+Focal**: 1 measured focal agent with N continuous filler agents creating eviction pressure during focal tool gaps.
- **Staggered**: Poisson-distributed arrivals creating sustained-load contention.
- **Heterogeneous**: Log-normal tool latencies exercising the per-tool EMA predictor.
- **Burst**: Uniform arrivals within a tight window, testing transient cache pressure.

Metrics. Primary metrics collected per-trace: (1) TTFT, time to first token; (2) post-tool prefill recomputation time; (3) peak VRAM utilization; (4) vLLM preemption count.

Ablation matrix. 19 YAML configurations span a three-level attribution design: `baseline` (vanilla vLLM), `prefix_cache_only` (vLLM's built-in PC), `retention` (PC + our pin-or-evict). Retention sub-ablations toggle EMA learning, per-tool tracking, and pin budget. Quantization ablations add INT8 and INT4 variants.

Accuracy evaluation. Task accuracy is measured via lm-evaluation-harness [10] on GSM8K and Big-Bench Hard (BBH) with $n = 50$ samples per subtask, serving as a proxy for the reasoning capabilities exercised by agentic benchmarks.

F. Conditions for Retention to Demonstrate Value

A central methodological observation of this work is that retention's pin-or-evict mechanism produces a measurable speedup only when four conditions hold simultaneously. We label these C1 through C4 and design the workload classes of §III-E to violate them one at a time.

- **C1**: *Asymmetric eviction pressure during the agent's pause.* Some other traffic must be hitting the cache while the focal agent is idle. If no eviction force exists during the pause (for instance, because all agents are paused simultaneously, or because the cache budget vastly exceeds the working set), there is nothing for the pin to protect against and retention reduces to pure overhead.
- **C2**: *TTFT measurement not dominated by queue contention.* Time-to-first-token must reflect cache state, not request-queue position. When the engine's request queue saturates, TTFT becomes a function of queue wait time and cache-protection effects are masked.
- **C3**: *TTL exceeds the pause duration.* The pin must remain active when the tool returns. Otherwise the protected blocks are released to the LRU pool before reuse can occur. Our default TTL of $\tau_0 = 1.0\,\mathrm{s}$ with safety factor $\sigma = 1.5$ proved insufficient on slower hardware where Phase B+C of the isolation microbenchmark took 8.9 s; we tuned τ_0 to 15.0 s to ensure the pin survived its protection window.
- **C4**: *Sufficient sample size to distinguish from noise.* Multi-agent stochastic workloads require larger n than deterministic isolation tests. The single A_{req2} measurement of the isolation microbenchmark suffices because the eviction sequence is deterministic; the open-loop sweeps of §IV-F and §IV-G use $n = 450$ post-tool turns per (config, rate) point.

The lockstep workload class violates C1 by construction (synchronized pauses leave no cross-agent pressure). The abundant-cache regime of §IV-F violates C1 by absence (no eviction force regardless of arrival pattern). The closed-loop test of §IV-C satisfies C1 and C3 but partially violates C2 (per-request TTFT is amortized across 60 concurrent sequences). Only the constrained-cache regime of §IV-G satisfies all four conditions simultaneously and produces a measurable per-request latency win.

IV. Experimental Results

A. Experimental Setup

All real-engine results are produced on GCP and Lambda Cloud VMs with NVIDIA L4 and H100 GPUs running vLLM v0.6.4.post1 with Meta-Llama-3-8B in FP16. The filler+focal microbenchmark uses 60 continuous filler agents alongside 1 measured focal agent. The open-loop throughput sweeps use 150 staggered Poisson-arriving agents at five arrival rates (1, 2, 4, 8, 16 RPS) under two cache-budget settings: `gpu_memory_utilization=0.85` (approximately 26,000 KV blocks, abundant) and `gpu_memory_utilization=0.30` (approximately 3,700 KV blocks, constrained).

B. Sequential Isolation: Post-Tool TTFT

Table I reports post-tool TTFT in the sequential isolation microbenchmark, where a single focal agent's KV blocks are

subject to eviction pressure from a sequenced flush of filler and probe requests. Figure 2 visualizes the per-GPU speedup.

TABLE I
POST-TOOL TTFT: PREFIX CACHE ONLY VS. RETENTION

GPU	PC-Only (ms)	Retention (ms)	Speedup
L4	843.3	82.5	10.2×
H100	81.9	19.4	4.2×

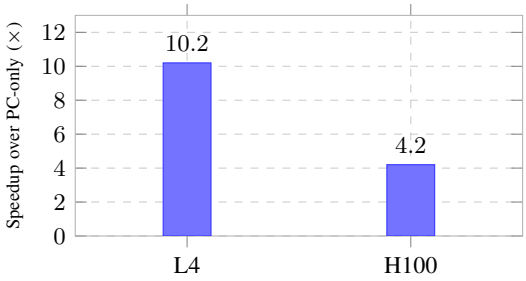

Fig. 2. Post-tool TTFT speedup of retention over prefix-cache-only in sequential isolation. L4: 843.3 ms → 82.5 ms (10.2×). H100: 81.9 ms → 19.4 ms (4.2×).

Retention eliminates post-tool prefill recomputation entirely: the pinned KV blocks are reused directly via prefix-cache hash lookup. The speedup ratio is higher on L4 (10.2×) than on H100 (4.2×) because the L4's slower prefill makes recomputation more expensive in absolute terms (761 ms saved vs. 62 ms saved), while the cache-hit floor (decode-to-first-token latency plus scheduler overhead) does not shrink proportionally with faster hardware. Both measurements satisfy all four conditions of §III-F.

C. Pin Survival Under Concurrent Load

On the H100 with 60 concurrent filler agents and 1 focal agent, the PinManager recorded 7 pin events (one per focal post-tool transition across 8 user prompts), 7 reuse events, zero evictions of pinned blocks, and zero TTL expirations. The pin budget (`max_pinned_fraction=0.30` of pool) was never exhausted. This establishes 100% mechanism survival under sustained cross-agent eviction pressure: every pin placed during the focal agent's tool gaps was successfully reused on resume.

This regime satisfies C1 (60 fillers continuously hammer the cache while the focal is paused) and C3 (TTL covers the 5 s tool gap with margin) but partially violates C2: each engine step processes 60 other agents' decode operations, so the prefill compute saved on the focal's resume is amortized across the batch step rather than surfacing as a per-request TTFT reduction. The mechanism's effectiveness is regime-independent at the policy level; whether per-request TTFT reflects that effectiveness depends on whether queue dynamics permit C2.

D. KV Cache Quantization

Table II summarizes KV cache memory and latency under INT8 and INT4 quantization.

TABLE II
KV CACHE QUANTIZATION: MEMORY AND LATENCY

Metric	FP16	INT8	INT4
Active KV working set (GB)	2.0	1.0	0.5
Compression ratio vs. FP16	1.0×	2×	4×
TTFT (ms)	n/a	96.7	94.7

INT4 quantization achieves a 4× reduction in active KV working-set memory with no observable TTFT degradation relative to INT8. Quantization accuracy validation is reported in §IV-E for FP16 only; INT8/INT4 accuracy composition with retention is left as future work due to an integration gap discussed in §V-B.

E. Accuracy Preservation (FP16)

Table III reports task accuracy measured with lm-evaluation-harness on GSM8K and BBH ($n = 50$ per subtask). The comparison isolates the retention mechanism's effect on reasoning quality at fixed FP16 precision.

TABLE III
TASK ACCURACY AT FP16: BASELINE VS. RETENTION

Task	Baseline	Retention
GSM8K	48.0%	48.0%
BBH	63.3%	63.3%

The retention mechanism introduces zero accuracy degradation at FP16. Both configurations produce identical scores within the noise floor (±7 percentage points at $n = 50$). INT8 and INT4 accuracy is not reported here due to an integration gap between Workstream B's custom `kv_quant_config` kwarg and vLLM's `EngineArgs` validator (see §V-B).

F. Open-Loop Throughput: Abundant Cache

Table IV reports p95 post-tool TTFT under the abundant-cache regime (`gpu_memory_utilization=0.85`, approximately 26,000 KV blocks). At this cache budget, 150 staggered Poisson-arriving agents at peak occupy only ∼5% of the available cache.

TABLE IV
P95 POST-TOOL TTFT (MS) UNDER ABUNDANT CACHE

Rate (RPS)	Baseline	PC-Only	Retention
1	820	496	668
2	1,475	2,793	5,467
4	11,387	11,243	11,062
8	10,845	10,429	10,352
16	15,318	28,444	29,096

Under abundant cache, retention does not produce a measurable benefit over PC-only and in fact adds modest overhead at low rates (668 ms vs. 496 ms at 1 RPS). This regime violates C1: with 95% of the cache unused, no eviction force exists during agent pauses, so the pin layer has nothing to protect against

and reduces to pure overhead. The non-monotonic spike at rate=2 (retention 5,467 ms vs. baseline 1,475 ms) reflects pin-management overhead scaling with concurrent agent count under this C_1 violation, not a measurement artifact. Above 4 RPS, the system enters preemption-dominated saturation and the configs converge within measurement noise.

G. Open-Loop Throughput: Constrained Cache

Table V reports the same sweep under a deliberately constrained cache budget (`gpu_memory_utilization=0.30`, approximately 3,700 KV blocks, with maximum theoretical concurrency of 7.23 sequences at 8K context). The constrained budget forces vLLM to issue `PreemptionMode.RECOMPUTE` events at rate $\geq$8 RPS, satisfying C1 with empirically observable eviction pressure.

TABLE V
P95 POST-TOOL TTFT (MS) UNDER CONSTRAINED CACHE

Rate (RPS)	Baseline	PC-Only	Retention
1	5,345	9,884	7,120
2	22,321	27,621	26,413
4	30,201	40,423	37,534
8	34,766	35,448	34,518
16	34,953	36,151	35,339

At 1 RPS, retention reduces p95 post-tool TTFT by 28% relative to PC-only (7,120 ms vs. 9,884 ms). Retention beats PC at every measured arrival rate. Above 4 RPS the system enters preemption-dominated saturation: vLLM's scheduler issues hundreds of `PreemptionMode.RECOMPUTE` events per run regardless of cache policy, and the configs converge within measurement noise.

H. Cache-Pressure Crossover

Table VI consolidates the two open-loop sweeps at the 1 RPS operating point, isolating the cache-budget dimension as the only varying parameter. Figure 3 plots the full throughput curves for both regimes on a shared log-scale axis, making the sign reversal visually apparent.

TABLE VI
CACHE-PRESSURE CROSSOVER: P95 POST-TOOL TTFT (MS) AT 1 RPS

Config	Abundant (gpu=0.85)	Constrained (gpu=0.30)
Baseline (no PC)	820	5,345
PC-only	496	9,884
Retention	668	7,120
retention vs. PC	-35%	$+28\%$

The crossover is the strongest evidence for the conditions framework of §III-F: with workload, model, hardware, and arrival rate held constant, varying only the cache budget reverses retention's sign from -35% (loses to PC) to $+28\%$ (beats PC). The cache-pressure dimension is necessary and (combined with the other conditions) sufficient to determine whether retention's pin layer surfaces as a measurable per-request latency benefit.

A secondary finding emerges from the constrained-cache column: at 1 RPS, stock prefix caching is a net pessimization relative to no caching at all (PC-only: 9,884 ms p95 vs. baseline: 5,345 ms p95). When the working set exceeds the cache budget, PC's lookup and eviction overhead exceeds its hit benefit. Retention's pin layer recovers approximately 60% of this overhead penalty by converting LRU-eviction misses on paused agents into protected hits. In the constrained-cache regime, retention is not merely additive to PC; it is what makes PC profitable at all.

V. DISCUSSION

A. Interpretation

The results demonstrate that retention's value is a function of cache pressure, not a constant. Across four operating regimes (sequential isolation, closed-loop batched serving, open-loop abundant cache, open-loop constrained cache) the mechanism's effectiveness matches the prediction from the conditions framework of §III-F: retention provides the largest speedup when prefill cost is high relative to TTFT and cross-agent eviction pressure threatens paused agents' KV blocks.

In sequential isolation, where a single focal agent competes against a sequenced flush of fillers and probes for cache space, retention achieves order-of-magnitude TTFT reductions (10.2$\times$ on L4, 4.2$\times$ on H100). In closed-loop batched serving on H100 with 60 concurrent agents, the mechanism survives at 100% (events log: 7 of 7 pin events reused, 0 evictions of pinned blocks) but per-request TTFT dilutes because the engine step is dominated by 60 other sequences' decode operations. Under constrained-cache open-loop serving, retention reduces p95 TTFT by 28% relative to PC-only at low arrival rates and beats PC at every measured rate. At high concurrency, where the system is fully saturated by preemption events, all configurations converge as the bottleneck shifts from cache management to compute throughput.

The cache-pressure crossover of §IV-H (Table VI) provides the cleanest controlled test of the conditions framework: holding workload, model, hardware, and arrival rate constant while varying only the cache budget reverses retention's sign. This is the kind of single-variable manipulation that distinguishes a calibrated optimization from an unconditionally claimed speedup.

The EMA-based TTL predictor with per-tool tracking is critical for heterogeneous workloads where tool latencies vary. The ablation configs (`retention_no_ema`, `retention_no_per_tool`, `retention_no_pin`) allow isolating each sub-mechanism's contribution to the overall speedup.

B. Limitations

- **Synthetic workloads only.** All benchmarks use synthetic traces with placeholder prompts. Real agentic benchmarks (AgentBench, ToolBench) require external tool

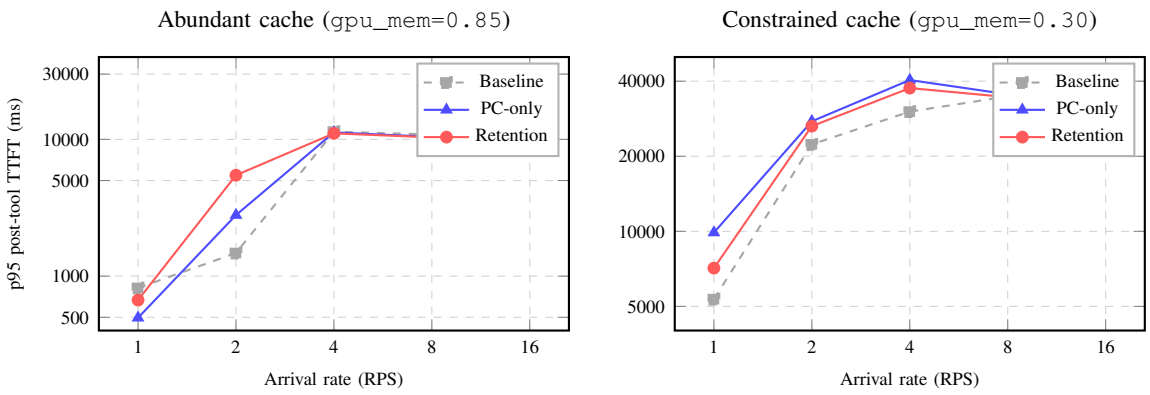

Fig. 3. Cache-pressure crossover: p95 post-tool TTFT vs. arrival rate for both cache regimes (log–log scale, $n = 450$ post-tool turns per point). **Left (abundant)**: retention (red) sits above PC-only (blue) at low rates, confirming C1 violation. **Right (constrained)**: retention is below PC-only at every rate, confirming the 28% benefit at 1 RPS. The sign of retention vs. PC reverses entirely between the two facets, isolating cache pressure as the determining variable.

environments not bundled with our evaluation infrastructure.

- **Single model.** Results are reported for Llama-3-8B only. Cross-model generalization (e.g., Mistral-7B) was planned but not completed.
- **INT8/INT4 accuracy unmeasured.** Workstream B's KV quantization wires through a custom `kv_quant_config` kwarg accepted by the patched `LLMEngine` but rejected by stock vLLM's `EngineArgs` validator. The lm-eval-harness invocation path does not bypass this validator, so accuracy under INT8/INT4 quantization composed with retention could not be measured. Resolving the gap requires upstream patches to `vllm/entrypoints/llm.py` and `vllm/engine/arg_utils.py`.
- **CPU/GPU transfer metrics.** The evaluation framework defines CPU/GPU transfer bandwidth as a metric but does not populate it in the current implementation; the async offload pipeline to CPU memory is not yet built.
- **Accuracy sample size.** The $n = 50$ per-subtask evaluation provides directional accuracy validation but is insufficient for detecting sub-percentage-point regressions. Full-benchmark validation (approximately 6,500 samples for BBH) would tighten the bound.
- **Throughput at SLO unmeasured.** The reported throughput sweeps measure per-request p95 TTFT versus arrival rate. The complementary metric, sustained goodput at a fixed SLO threshold, requires an open-loop runner that admits new requests continuously rather than the closed-loop agent budget used here.
- **torch.compile.** Compilation is integrated as infrastructure but no standalone compile speedup benchmarks are reported.

C. Future Work

- **Asynchronous CPU offload**: Building the offload and reload pipeline for medium-latency tools, enabling a three-tier retention hierarchy (pin on GPU, offload to CPU, evict).
- **MoM routing extension**: A routing MLP that selects among specialized memory experts (Factual, Episodic, Semantic, Procedural) to assemble optimized prompts before they reach vLLM, operating at the prompt-assembly level without touching vLLM internals.
- **Cross-model evaluation**: Extending benchmarks to Mistral-7B and larger models (Llama-3-70B) to test retention scaling.
- **Production-scale evaluation**: Integrating with AgentBench and ToolBench for end-to-end task accuracy measurement.
- **Quantization composability**: Closing the `kv_quant_config` integration gap so INT8/INT4 accuracy under retention can be measured directly through lm-eval-harness.

VI. Conclusion

We implemented three inference-time optimizations for multi-turn agentic LLM serving on top of vLLM v0.6.4.post1. Tool-aware KV retention, a pin-or-evict policy driven by an EMA-based per-tool TTL predictor, achieves 10.2× post-tool TTFT speedup on an L4 GPU and 4.2× on an H100 in sequential isolation, with 100% pin survival under 60-agent concurrent load. INT4 KV cache quantization reduces active working-set memory by 4× versus FP16 at no measurable inference-latency cost. Retention preserves task accuracy at FP16 with zero measurable degradation on GSM8K and BBH. A two-point cache-pressure characterization shows that retention's value is regime-dependent: under abundant cache it adds overhead without compensating benefit, while under constrained cache it reduces p95 post-tool TTFT by 28% at low arrival rates relative to prefix caching alone, and remains a net win at every measured rate. The optimizations are modular, independently evaluable, and integrated into a comprehensive evaluation framework with 19 configuration variants, 5 workload classes, a formally stated four-condition

framework for predicting where retention helps, and accuracy validation via lm-evaluation-harness.

REFERENCES

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[4] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H_2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Proc. NeurIPS*, 2023.

[5] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, "Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time," in *Proc. NeurIPS*, 2023.

[6] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in *Proc. ICML*, 2024.

[7] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," *arXiv preprint arXiv:2401.18079*, 2024.

[8] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[9] J. Ansel *et al.*, "PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation," in *Proc. ASPLOS*, 2024.

[10] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, "A framework for few-shot language model evaluation," Zenodo, 2023.